

DVWA Security Lab Report

Damn Vulnerable Web Application — Low Security Level

Naveen Ushan | University of Colombo School of Computing

Labs covered: SQL Injection · Brute Force · Command Injection · File Upload · CSRF

SQL Injection — Low Security Level

2. I trying to exploit a vulnerable SQL query by modifying the input to retrieve unauthorized user information from the database.

3. Steps

1. Logged into DVWA.
2. Navigated to DVWA Security → Set Security Level → Low.
3. Opened Vulnerabilities → SQL Injection.
4. Entered the normal input to confirm expected behaviour:

```
1
```

5. Clicked Submit and observed the details of the user with ID 1.
6. Entered the SQL injection payload:

```
1' OR '1'='1
```

7. Clicked Submit. The application returned multiple user records because the input changed the original SQL query logic.

4. Screenshot(s)

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)[PHP Info](#)[About](#)

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>



- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection**
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API

- DVWA Security
- PHP Info
- About

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

5. This demonstrates that the application is vulnerable to SQL Injection because user input is included directly in SQL queries without proper validation, allowing an attacker to manipulate the query and access unauthorized database records.

6. Use parameterized queries (prepared statements) instead of directly inserting user input into SQL statements, and enforce input validation and restricted database user privileges.

Brute Force — Low Security Level

2. I trying to Bypass the login authentication by repeatedly trying different password combinations until finding valid credentials.

3. Steps

8. Logged into DVWA.
9. Changed the security level to Low.
10. Navigated to Vulnerabilities → Brute Force.
11. Entered a username:

admin
12. Tested different passwords manually and using a brute-force attack tool.
13. Used the correct password:

Password 123
14. Submitted the login request. The system accepted the credentials and displayed a successful login message.

4. Screenshot(s)



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

5. This proves that the application has no protection against repeated login attempts, allowing an attacker to guess passwords and gain access to user accounts.
6. Implement account lockout mechanisms, rate limiting, CAPTCHA, and multi-factor authentication, along with enforced strong password policies.

Command Injection — Low Security Level

2. I trying to execute unauthorized operating system commands by injecting additional commands into a vulnerable input field.

3. Steps

15. Logged into DVWA.

16. Set the security level to Low.

17. Opened Vulnerabilities → Command Injection.

18. Entered the normal IP address:

```
127.0.0.1
```

19. Submitted the request and observed the normal ping response.

20. Entered the command injection payload (Windows):

```
127.0.0.1 & whoami
```

21. Also tested:

```
127.0.0.1 & dir
```

22. Submitted the request. The server executed the additional operating system command.

4. Screenshot(s)



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection**
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
Pinging 127.0.0.1 with 32 bytes of data:  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128  
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
```

```
Ping statistics for 127.0.0.1:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
Approximate round trip times in milli-seconds:  
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection**
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
```

```
Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
Volume in drive C is OS
Volume Serial Number is 1008-C73B
```

```
Directory of C:\xampp\htdocs\DVWA\vulnerabilities\exec
```

```
07/11/2026  12:41 AM
```

```
07/11/2026  12:41 AM
```

```
07/11/2026  12:41 AM
```

```
help
```

```
07/11/2026  12:41 AM
```

```
1,829 index.php
```

```
07/11/2026  12:41 AM
```

```
source
```

```
1 File(s)
```

```
1,829 bytes
```

```
4 Dir(s) 639,679,258,624 bytes free
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

5. This proves that the application allows users to execute operating system commands through unsanitized input, meaning an attacker could run malicious commands on the server.

6. Avoid executing system commands using user input; use secure APIs, validate input against allowlists, and restrict user permissions.

File Upload — Low Security Level

2. I'm trying to upload a PHP file containing executable code because the application does not properly validate uploaded files.

3. Steps

23. Logged into DVWA.

24. Set the security level to Low.

25. Opened Vulnerabilities → File Upload.

26. Created a PHP file named shell.php with the following code:

```
<?php echo " Check the project"; ?>
```

27. Uploaded the file through the File Upload page. The application accepted the PHP file without checking the file type.

28. Opened the uploaded file location:

```
http://localhost:8001/DVWA/hackable/uploads/shell.php
```

29. The page displayed: "DVWA Upload Successful".

4. Screenshot(s)

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

DVWA

Vulnerability: File Upload

The PHP module **GD** is not installed.

Choose an image to upload:

Choose File

No file chosen

Upload

../../../../hackable/uploads/shell.php succesfully uploaded!

More Information

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- <https://www.acunetix.com/websitesecurity/upload-forms-threat/>

← ↻ 🏠 ⓘ

localhost:8001/DVWA/hackable/uploads/shell.php

Check the project

5. This demonstrates that the application allows unrestricted file uploads, meaning an attacker could upload malicious files and execute code on the server.

6. Restrict allowed file extensions, validate MIME types, rename uploaded files, store uploads outside the web root, and ensure uploaded files are not executable.

Cross-Site Request Forgery (CSRF) — Low Security Level

2. I trying to change a user's password without confirming the previous password by exploiting missing CSRF protection.

3. Steps

30. Logged into DVWA.

31. Changed the security level to Low.

32. Opened Vulnerabilities → CSRF.

33. Entered a new password:

Password123

34. Confirmed the password:

Password123

35. Clicked Change. The password was changed successfully without asking for the old password or verifying the request source.

4. Screenshot(s)



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF**
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Password Changed.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

5. This proves that the application is vulnerable to CSRF because it accepts sensitive actions without verifying whether the request was intentionally made by the user.

6. Implement CSRF tokens for all sensitive actions, verify request origins, and use secure cookie settings such as the SameSite attribute.